HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

PROJECT 01

HỆ ĐA AGENT CHO BỆNH TIỂU ĐƯỜNG

PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

Giảng viên:	PGS. TS. Trần Đình Quế
Sinh viên:	Kiều Trường Giang
Mã sinh viên:	B23DCCN254
Nhóm học phần:	06
Nhóm thực hiện:	02

Hà Nội, tháng 11 năm 2026

Mục lục

1 Giới thiệu **1**

1.1 Bài toán và sản phẩm 1

1.2 Phạm vi đầu ra 1

1.3 Mục tiêu đánh giá 1

2 Agent và cơ chế phối hợp **2**

2.1 Vai trò xử lý 2

2.2 Reinforcement Learning kết hợp luật 2

2.3 Giới hạn của kiến trúc 3

3 Kiến trúc tổng quan **4**

3.1 Nhánh dữ liệu 4

3.2 Nhánh tri thức 4

3.3 Luồng người dùng 5

4 Các thành phần và thực nghiệm **6**

4.1 Phân loại 6

4.2 Hồi quy 6

4.3 Kết quả kiểm tra 7

4.4 Truy xuất và đồ thị 7

4.5 Phân tích sai số theo lớp 7

4.6 Phân tích phần dư hồi quy 9

4.7 Hợp đồng dữ liệu và API 9

4.8 Kết quả kiểm thử tích hợp 9

4.9 Đối chiếu với sơ đồ yêu cầu 9

5 Code và demo **11**

5.1 Địa chỉ và cách chạy 11

5.2 Kịch bản trình diễn 11

5.3 Hạn chế và hướng hoàn thiện 12

5.4 Ảnh chụp sản phẩm đang chạy 14

A Mã nguồn tái lập **18**
A.1 scripts/ingest_graph.py 18
A.2 scripts/train_models.py 18
A.3 scripts/train_policy.py 19
A.4 app/agents.py 20
A.5 app/main.py 20
A.6 scripts/test_scenarios.py 22

Chương 1

Giới thiệu

1.1 Bài toán và sản phẩm

Project xây dựng GlucoGuide, một ứng dụng web tích hợp hai nhánh xử lý của sơ đồ học phần. Nhánh dữ liệu sử dụng mô hình mạng nơ-ron nhiều lớp để phân loại nhãn nguy cơ tổng hợp và dự đoán điểm tiến triển bệnh trong dữ liệu nghiên cứu. Nhánh tri thức kế thừa kho tài liệu của tiểu luận 02, tổ chức các khái niệm cùng quan hệ trên Neo4j và truy xuất văn bản theo câu hỏi. Kết quả hai nhánh được trình bày cùng nguồn bằng chứng và nhật ký xử lý.

Sinh viên thực hiện là Kiều Trường Giang, B23DCCN254, nhóm học phần 06, nhóm thực hiện 02. Project được phát triển từ các sản phẩm của sinh viên, có tham khảo bố cục và cách mô tả kiến trúc trong báo cáo của Nguyễn Tự Kiên. Các số liệu của bài tham khảo không được dùng làm kết quả của project này.

1.2 Phạm vi đầu ra

Hai bộ dữ liệu có định nghĩa nhãn khác nhau. Bộ phân loại gồm các nhãn High, Moderate và Low của dữ liệu tổng hợp. Bộ hồi quy gồm những người đã mắc bệnh và một thước đo tiến triển sau một năm. Vì vậy không thể ghép một nhãn High với một điểm hồi quy để suy ra xác suất mắc bệnh. Ứng dụng dùng hai biểu mẫu riêng và công bố phạm vi ngay tại đầu ra.

Sản phẩm hiện sử dụng tổng hợp trích đoạn xác định. Câu trả lời được ghép từ các đoạn văn truy xuất được và thông tin về phạm vi mô hình; không gọi mô hình ngôn ngữ bên ngoài. Lựa chọn này giúp kiểm tra nguồn và tái lập kết quả, nhưng khả năng đối thoại tự nhiên bị giới hạn bởi từ vựng và độ phủ của kho tài liệu.

1.3 Mục tiêu đánh giá

Phân loại được đo bằng accuracy và macro-F1. Hồi quy được đo bằng MAE và hệ số xác định. Truy xuất cần kiểm tra cả tài liệu được chọn và quan hệ có nguồn trên Neo4j. Kiểm tra ứng dụng tập trung vào các tình huống có bằng chứng, thiếu bằng chứng, thiếu đầu vào, giá trị không hợp lệ và cơ sở dữ liệu đồ thị không hoạt động.

Chương 2

Agent và cơ chế phối hợp

2.1 Vai trò xử lý

Hệ thống phân chia trách nhiệm thành tiếp nhận, kiểm tra dữ liệu, mô hình, tri thức, chính sách lựa chọn bước phản hồi, đối chiếu phạm vi và phản hồi. Bản triển khai hiện điều phối các vai trò này trong một tiến trình FastAPI. Nhật ký lưu người gửi, người nhận, loại thông điệp và nội dung. Đây là mô hình điều phối tập trung; các agent được gọi qua MessageBus với handler và bộ nhớ trong lượt, chưa chạy như những dịch vụ tự chủ độc lập.

Một thông điệp REQUEST thể hiện yêu cầu xử lý. INFORM ghi nhận kết quả và FAILURE thể hiện thành phần không thực hiện được. Khi Neo4j không hoạt động, hệ thống ghi FAILURE và chỉ dùng các đoạn văn cục bộ, không giả lập kết quả đồ thị. Khi thiếu đầu vào mô hình, hệ thống trả lại danh sách trường cần bổ sung trước khi suy luận.

2.2 Reinforcement Learning kết hợp luật

Chính sách được học trong môi trường mô phỏng luồng xử lý, không mô phỏng hiệu quả điều trị. Trạng thái gồm hai biến nhị phân: đã có kết quả mô hình và đã có bằng chứng truy xuất. Bốn trạng thái tương ứng với bốn tổ hợp của hai biến này. Ba hành động là hỏi làm rõ, trả lời có nguồn và giải thích mô hình kèm nguồn.

Hàm thưởng bằng một nếu hành động phù hợp trạng thái, bằng âm một trong trường hợp còn lại. Khi chưa có bằng chứng, hành động mục tiêu là hỏi làm rõ. Khi có bằng chứng nhưng chưa có kết quả mô hình, hành động mục tiêu là trả lời có nguồn. Khi có cả hai, hành động mục tiêu là giải thích mô hình và dẫn nguồn. Đây là hàm thưởng do người thiết kế quy định; không phải đánh giá của bệnh nhân hay chuyên gia.

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r - Q(s, a)].$$

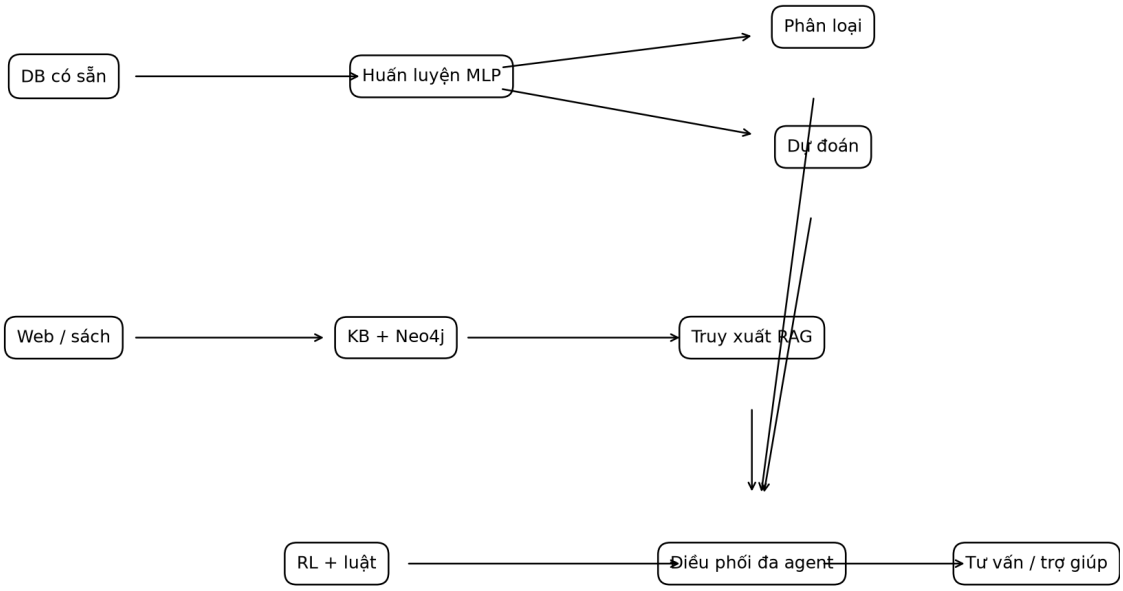
Mỗi episode kết thúc sau một hành động, nên không có thành phần bootstrap tương lai. Cấu hình gồm 3.000 episode, seed 254, tốc độ học 0,15 và xác suất khám phá 0,2. Vì bài toán chỉ có một bước, thí nghiệm minh họa cơ chế học giá trị hành động nhưng chưa chứng minh khả năng lập kế hoạch dài hạn. Luật thiếu bằng chứng luôn ép hành động hỏi làm rõ, kể cả khi chính sách học đề xuất hành động khác.

2.3 Giới hạn của kiến trúc

Việc hiển thị nhiều vai trò trong nhật ký không tự chứng minh tính tự chủ của một hệ đa agent. Bản hiện tại đã có handler và bộ nhớ theo lượt; hướng mở rộng là bộ nhớ hội thoại bền vững, cơ chế chất vấn khi có bất đồng và triển khai dịch vụ độc lập. Báo cáo phân biệt rõ những vai trò đã điều phối được với các năng lực còn cần phát triển.

Chương 3

Kiến trúc tổng quan



Hình 3.1: Hai nhánh hội tụ về điều phối và trợ giúp, dựng lại từ sơ đồ trên bảng.

3.1 Nhánh dữ liệu

DB có sẵn cung cấp dữ liệu cho quá trình huấn luyện. Pipeline phân loại gồm điền trung vị, chuẩn hóa và MLP. Pipeline hồi quy chuẩn hóa dữ liệu rồi suy luận bằng MLP. Hai đầu ra đều đi tới vùng phản hồi của ứng dụng. Trong lúc phục vụ, mô hình được nạp từ checkpoint; ứng dụng không huấn luyện lại theo câu hỏi của người dùng.

3.2 Nhánh tri thức

Nguồn tri thức là các đoạn văn đã được tổ chức trong tiểu luận 02. Mỗi tài liệu có mã, tiêu đề, từ khóa, văn bản, định danh nguồn và các bộ ba. Project bảo toàn những trường này khi chuyển

sang kho tri thức mới. Việc cập nhật hướng dẫn y tế từ web chưa tự động hóa; phiên bản kho tài liệu phải được xem như một snapshot cần rà soát trước khi mở rộng phạm vi sử dụng.

Một bộ ba gồm khái niệm đầu, tên quan hệ và khái niệm cuối. Neo4j lưu hai đầu dưới nhãn Concept; quan hệ RELATED_TO có thuộc tính label biểu diễn tên quan hệ, document biểu diễn tài liệu và evidence biểu diễn đoạn văn làm căn cứ. Ràng buộc duy nhất theo tên khái niệm và thao tác MERGE giúp nạp lại dữ liệu mà không tạo các bản sao giống hệt.

3.3 Luồng người dùng

Câu hỏi kiến thức được chuyển đến truy xuất. Yêu cầu mô hình phải đi qua kiểm tra các trường trước khi suy luận. Văn bản và kết quả đồ thị được chuyển sang đối chiếu; chính sách học và luật xác định kiểu phản hồi. Giao diện hiển thị câu trả lời, tài liệu nguồn và các bước xử lý. Các trường của hai mô hình được tách riêng để tránh biến đổi sai đơn vị hoặc tự suy diễn giá trị còn thiếu.

Chương 4

Các thành phần và thực nghiệm

4.1 Phân loại

Dữ liệu tổng hợp gồm 50.000 dòng. Tám biến đầu vào là tuổi, BMI, HbA1c, đường huyết lúc đói, glucose, huyết áp tâm thu, HDL và LDL. Các biến chứa điểm nguy cơ, khuyến nghị sức khỏe và yêu cầu gặp bác sĩ không được đưa vào mô hình. Phép chia phân tầng với seed 254 giữ 40.000 dòng huấn luyện và 10.000 dòng kiểm tra.

MLP có ba tầng ẩn 64, 32 và 16 đơn vị, dừng sớm và tối đa 150 vòng lặp. Trung vị và tham số chuẩn hóa được học trên tập huấn luyện ngoài. Cơ chế early stopping của thư viện chia tiếp một phần dữ liệu cho validation; do tiền xử lý nằm trước MLP, các tham số tiền xử lý đã nhìn thấy phần validation nội bộ này. Tập test ngoài vẫn độc lập, nhưng để kiểm soát validation nghiêm ngặt hơn cần tách validation trước toàn bộ pipeline.

Logistic Regression dùng trọng số lớp cân bằng làm đối chứng. MLP dùng cấu hình mặc định không cân bằng trọng số. Vì hai cấu hình có cách xử lý mất cân bằng khác nhau, chênh lệch kết quả không chỉ phản ánh kiến trúc mạng. Accuracy cần đọc cùng macro-F1, đặc biệt với lớp Low hiếm.

4.2 Hồi quy

Bộ diabetes từ scikit-learn được tải ở dạng chưa chuẩn hóa, gồm 442 quan sát với 10 biến đầu vào. Phép chia seed 254 giữ 353 mẫu huấn luyện và 89 mẫu kiểm tra. Ridge với hệ số điều chuẩn 10 là đối chứng. MLP có ba tầng ẩn 64, 32 và 16, dừng sớm và tối đa 1.000 vòng lặp.

Các biến s1 đến s6 là những trường nghiên cứu của bộ dữ liệu. Không được thay tên hoặc đơn vị bằng suy đoán khi nhập hồ sơ ngoài đời. Mục tiêu của demo là minh họa quy trình suy luận từ mô hình đã lưu. Chất lượng trên bộ dữ liệu nhỏ không chứng minh khả năng chuyển sang quần thể Việt Nam.

4.3 Kết quả kiểm tra

Mô hình	Accuracy/MAE	F1/R2	Mẫu test
LogisticRegression	0.6965	0.5005	10000
MLPClassifier	0.8180	0.5441	10000
Ridge	44.0555	0.4791	89
MLPRegressor	44.3225	0.4758	89

MLP cải thiện accuracy và macro-F1 so với Logistic Regression trong lần chạy này. Tuy nhiên macro-F1 còn thấp so với accuracy, phản ánh hiệu năng không đồng đều giữa các lớp. Ở hồi quy, Ridge có MAE thấp hơn và hệ số xác định cao hơn MLP. Báo cáo vẫn trình diễn MLP để minh họa nhánh Deep Learning, đồng thời công bố đầy đủ đối chứng và không nhận MLP là mô hình tốt nhất.

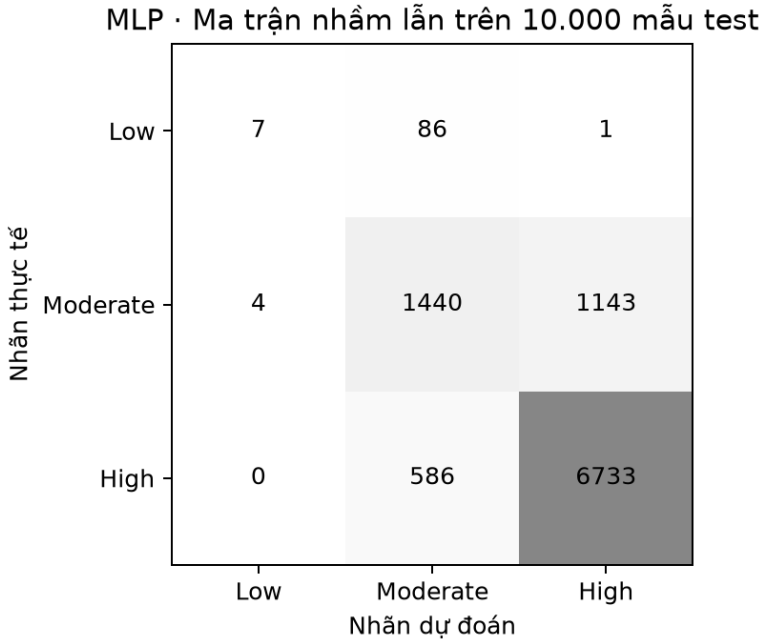
4.4 Truy xuất và đồ thị

Văn bản được biểu diễn bằng TF-IDF với unigram và bigram, bỏ dấu khi chuẩn hóa. Câu hỏi được biến đổi bằng cùng từ vựng. Điểm truy xuất là tích vô hướng của các vector đã chuẩn hóa. Hệ thống giữ tối đa ba đoạn có điểm lớn hơn giá trị lớn nhất giữa 0,04 và 65% điểm của đoạn đứng đầu. Ngưỡng này là quy tắc triển khai chưa được chọn trên một bộ validation độc lập.

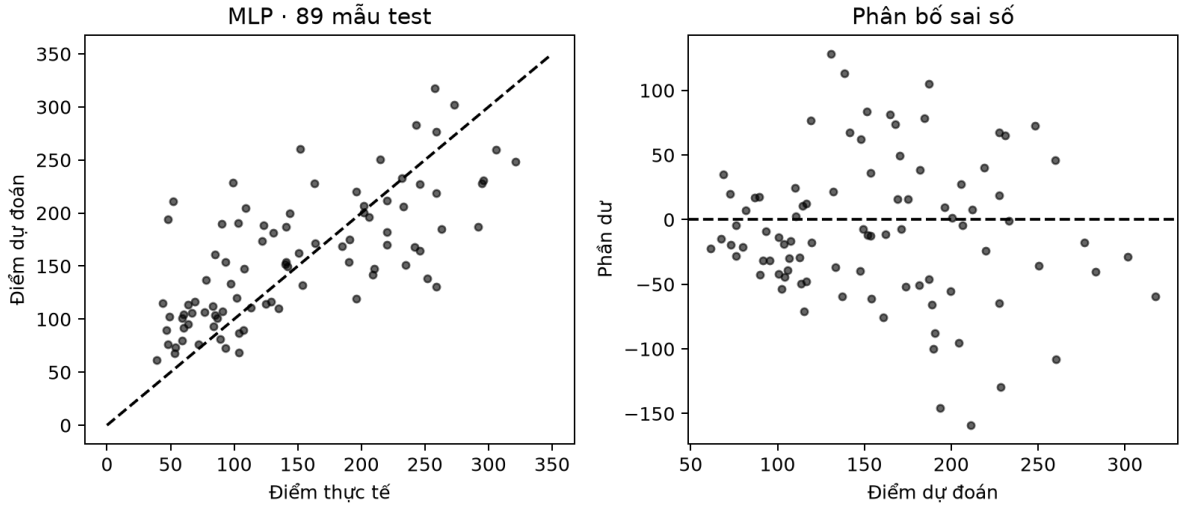
Neo4j lấy các cạnh có thuộc tính document trùng với mã tài liệu đã truy xuất. Do đó đồ thị cung cấp cấu trúc và căn cứ bổ sung cho truy xuất văn bản; đây chưa phải một bộ máy suy luận đa bước tổng quát. Một nhãn quan hệ không chứng minh quan hệ nhân quả. Khi không có tài liệu đủ điểm, hệ thống yêu cầu người dùng làm rõ.

4.5 Phân tích sai số theo lớp

Ma trận nhầm lẫn được tạo từ toàn bộ 10.000 dự đoán ngoài mẫu, không lấy từ tập huấn luyện. Tập test có 7.319 mẫu High, 2.587 mẫu Moderate và 94 mẫu Low. MLP dự đoán đúng 6.733 mẫu High, 1.440 mẫu Moderate nhưng chỉ 7 mẫu Low. Vì vậy accuracy 81,8% che giấu khả năng nhận diện lớp hiếm còn yếu. Recall của Low chỉ khoảng 7,45%; đây là hạn chế cần nêu khi bảo vệ báo cáo. Tỷ lệ lỗi giữa Moderate và High cho thấy ranh giới của nhãn tổng hợp chưa được mô hình tách tốt. Cải thiện tiếp theo cần so sánh trọng số lớp, điều chỉnh ngưỡng và cách lấy mẫu trên validation, đồng thời giữ nguyên test để tránh chọn cấu hình theo kết quả test. Không đổi tên nhãn tổng hợp thành chẩn đoán y tế.



Hình 4.1: Ma trận nhầm lẫn của MLP trên tập kiểm tra phân tầng.



Hình 4.2: Dự đoán và phần dư của MLP trên 89 mẫu kiểm tra.

4.6 Phân tích phần dư hồi quy

Đường chéo thể hiện dự đoán bằng giá trị thật. Những điểm cách xa đường chéo đóng góp lớn vào MAE. Phần dư được tính bằng giá trị thực trừ dự đoán; phần dư dương biểu thị dự đoán thấp hơn thực tế. Không thể dùng một chỉ số tổng hợp để bảo đảm mọi nhóm đầu vào có sai số tương đương. Bản này chưa xây dựng khoảng dự báo và chưa hiệu chỉnh bất định.

4.7 Hợp đồng dữ liệu và API

GET /api/schema cung cấp thứ tự đặc trưng. GET /api/examples cung cấp một mẫu từ mỗi bộ dữ liệu để trình diễn. POST /api/chat nhận question, task và values. Với task knowledge, hệ thống truy xuất văn bản; với classification hoặc regression, values phải đủ các trường tương ứng. Kết quả gồm answer, result, sources, graph_evidence và trace.

Trường result của phân loại gồm label, scores và scope. scores là đầu ra xác suất lớp của mô hình trên nhãn tổng hợp, chưa được hiệu chỉnh. Trường result của hồi quy gồm value và scope. Việc tách cấu trúc này tránh dùng chung một con số cho hai ý nghĩa khác nhau. Dữ liệu mẫu là mẫu của bộ nghiên cứu, không phải hồ sơ do người dùng cung cấp.

Kiểm tra đầu vào hiện yêu cầu số hữu hạn, giới hạn trị tuyệt đối và đủ số trường. Đây là kiểm tra kỹ thuật, chưa phải kiểm định lâm sàng từng đơn vị. Giá trị nhập bằng số nhưng sai đơn vị vẫn có thể làm kết quả vô nghĩa; vì vậy demo được giới hạn ở dữ liệu nghiên cứu và có nút điền mẫu.

4.8 Kết quả kiểm thử tích hợp

Script test_scenarios.py thực hiện tám trường hợp: câu hỏi có nguồn; câu hỏi ngoài từ vựng; suy luận phân loại; suy luận hồi quy; thiếu trường; giá trị không hữu hạn; tên bài toán không hợp lệ; lỗi Neo4j có kiểm soát. Tám trường hợp đều đạt điều kiện kiểm tra ở lần chạy lưu trong results/api_scenarios.json. Kiểm tra lỗi cơ sở dữ liệu được thực hiện bằng thay thế driver trong test, không phải chủ động tắt cơ sở dữ liệu đang phục vụ người dùng.

Kiểm thử xác nhận hợp đồng phản hồi và nhánh lỗi, chưa đo độ chính xác truy xuất trên tập câu hỏi lớn. Trường hợp ngoài miền sử dụng chuỗi không có trong từ vựng, do đó không chứng minh hệ thống phát hiện mọi câu hỏi ngoài miền bằng tiếng Việt. Những giới hạn này giúp tách kiểm thử chức năng khỏi đánh giá chất lượng tư vấn.

4.9 Đối chiếu với sơ đồ yêu cầu

Thành phần	Hiện thực trong sản phẩm

DB và Deep Learning	Hai bộ dữ liệu tách biệt, MLPClassifier và MLPRegressor; có checkpoint và kết quả test.
Phân loại và dự đoán	Hai lựa chọn trong phòng thực nghiệm, cùng chuyển kết quả đến phản hồi và nhật ký.
Web/sách đến KB	Kế thừa KB đã biên soạn từ tiểu luận 02; bảo toàn mã nguồn tài liệu, không tự động thu thập web ở thời điểm chạy.
KB và RAG	TF-IDF chọn đoạn văn; Neo4j trả các quan hệ cùng mã tài liệu; phản hồi trích đoạn có nguồn.
Agent và RL + rule	Handler và MessageBus xử lý yêu cầu; chính sách học một bước chọn kiểu phản hồi, luật chặn thiếu bằng chứng.
Tư vấn, trợ giúp	Chatbot giải thích kiến thức và phạm vi đầu ra; chưa sinh đối thoại tự do bằng LLM.

Chương 5

Code và demo

5.1 Địa chỉ và cách chạy

Demo cục bộ: `http://localhost:8765`. Neo4j Browser: `http://localhost:7474/browser/`. Repository đích: `https://github.com/jellalaz/He_Thong_Thong_Minh_Thay_Que`. Demo công khai tại thời điểm hoàn thiện: `https://bear-parental-pair-mercury.trycloudflare.com`. Đây là đường hầm tạm thời, phụ thuộc máy và tiến trình đang chạy; không phải hosting thường trực. Địa chỉ localhost chỉ hoạt động trên máy đang chạy dịch vụ. Báo cáo không coi địa chỉ này là một website công khai.

Cài các thư viện trong requirements.txt vào môi trường riêng. Chạy scripts/train_models.py để tạo checkpoint, scripts/train_policy.py để tạo chính sách và scripts/ingest_graph.py để nạp đồ thị. Sau đó chạy uvicorn app.main:app từ thư mục project. Cấu hình Neo4j nằm trong .env và không đưa lên repository.

5.2 Kịch bản trình diễn

Kịch bản thứ nhất: hỏi "HbA1c là gì?". Kiểm tra câu trả lời có mã tài liệu, kiểm tra bước truy xuất và số quan hệ Neo4j. Kịch bản thứ hai: hỏi một chủ đề không có trong kho, chẳng hạn lịch sử một đội bóng. Hệ thống phải yêu cầu làm rõ hoặc báo thiếu bằng chứng, không tự tạo nguồn.

Kịch bản thứ ba: nhập đủ tám trường phân loại bằng một mẫu của dữ liệu. Ghi lại nhãn, thông báo phạm vi và nhật ký mô hình. Kịch bản thứ tư: nhập đủ mười trường hồi quy từ mẫu nghiên cứu, kiểm tra đầu ra số và thông báo đây là điểm tiến triển. Hai kịch bản không dùng chung biểu mẫu.

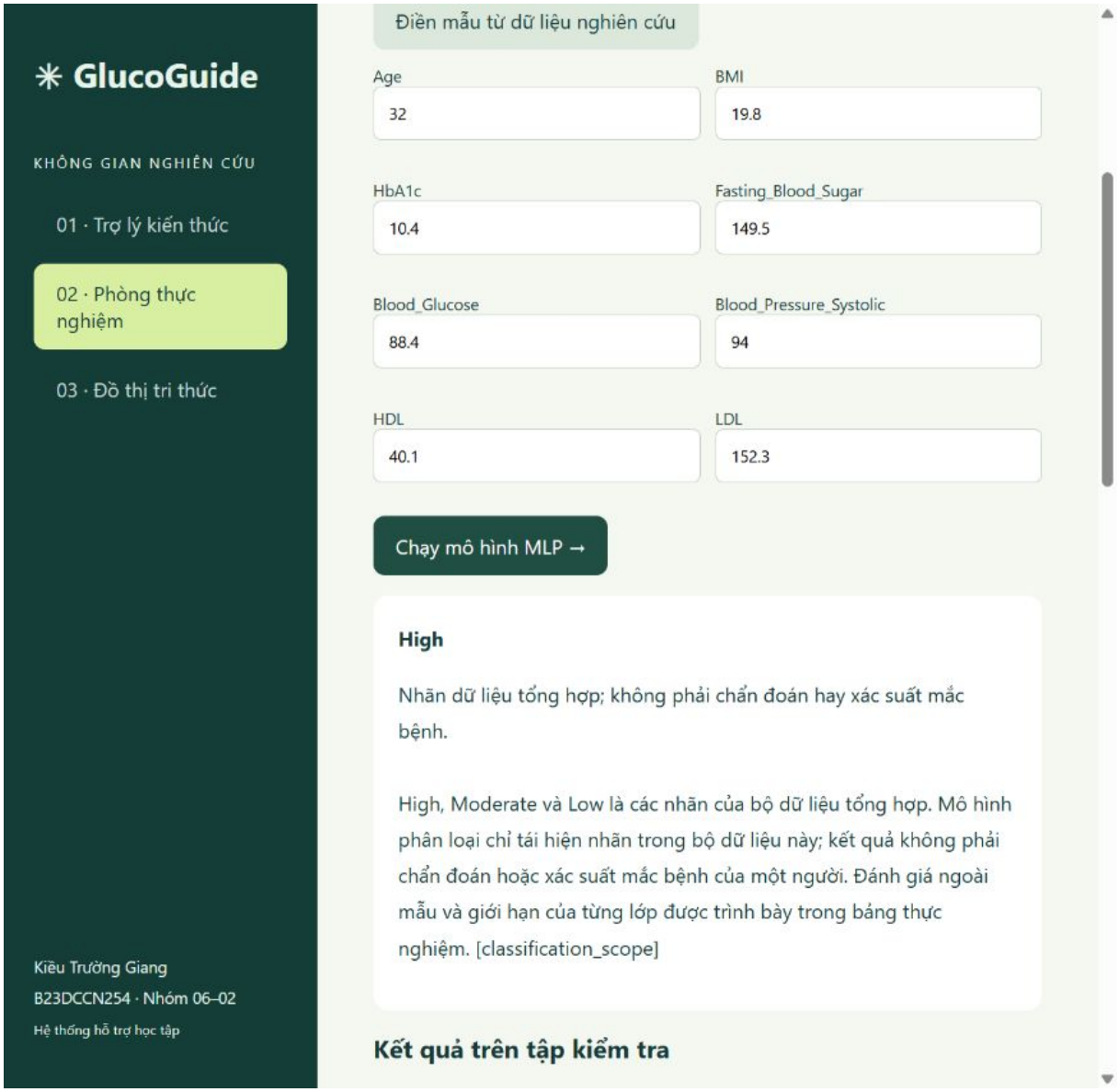
Kịch bản thứ năm: bỏ một trường bắt buộc. Hệ thống phải nêu trường còn thiếu. Kịch bản thứ sáu: nhập giá trị không hữu hạn hoặc chuỗi ký tự vào API, hệ thống phải từ chối. Kịch bản thứ bảy: tắt Neo4j rồi hỏi kiến thức, kiểm tra nhật ký FAILURE và việc công bố chỉ dùng văn bản. Kịch bản thứ tám: mở Neo4j Browser và thực hiện truy vấn dưới đây để đối chiếu với giao diện đồ thị.

```
MATCH (a:Concept)-[r:RELATED_TO]->(b:Concept)
RETURN a,r,b LIMIT 100;
```

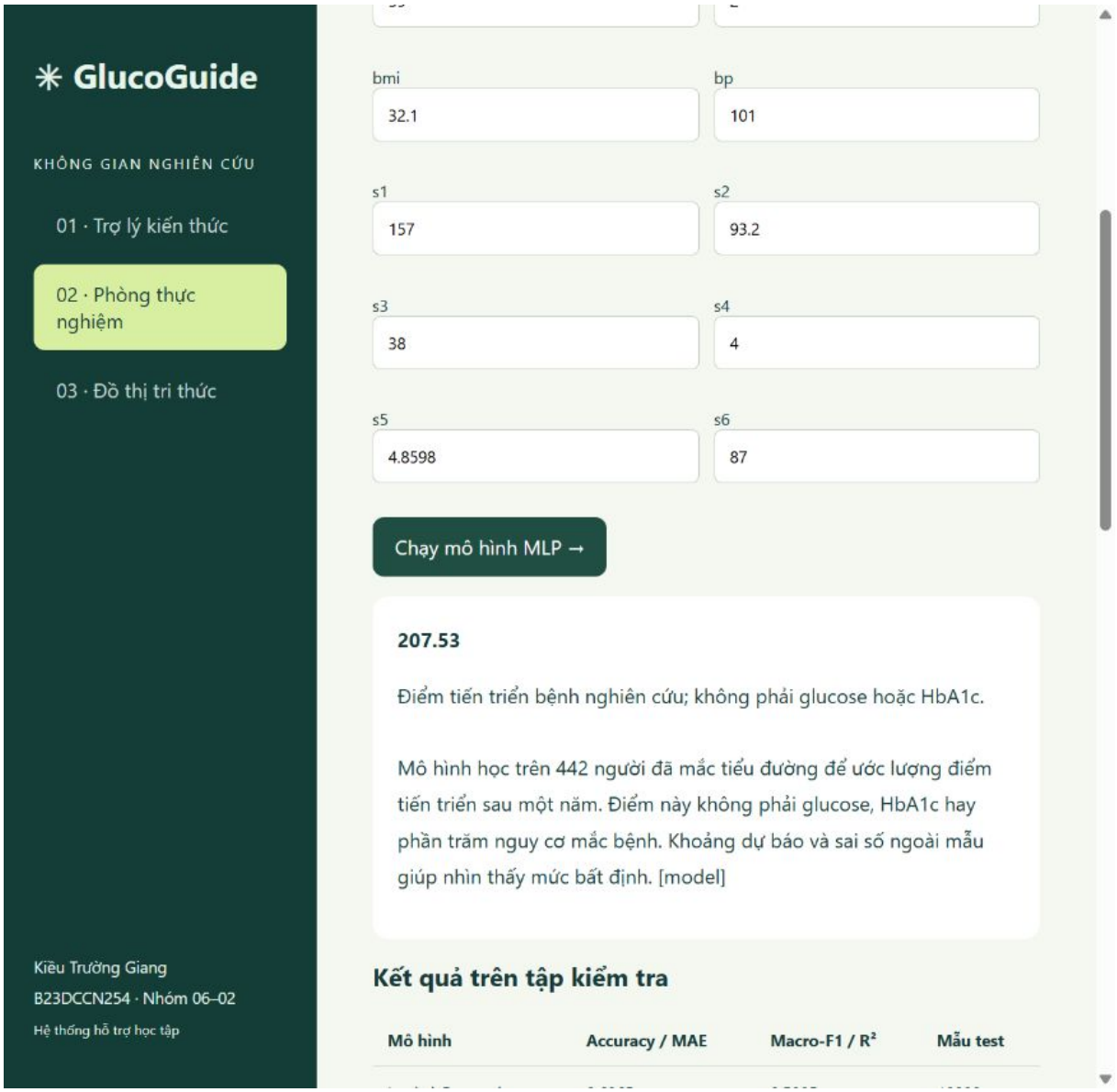
5.3 Hạn chế và hướng hoàn thiện

Bản hiện tại chưa có bộ nhớ hội thoại, chưa đọc phiếu xét nghiệm, chưa dùng LLM để diễn đạt và chưa có hội đồng tranh biện độc lập. Các phần này không được nhận là đã hoàn thành. Kiểm chứng mô hình mới chỉ dùng một seed; chưa có đánh giá chuyên gia, chưa có dữ liệu ngoài miền và chưa kiểm thử tải lớn. RL học quy trình mô phỏng, không quyết định điều trị.

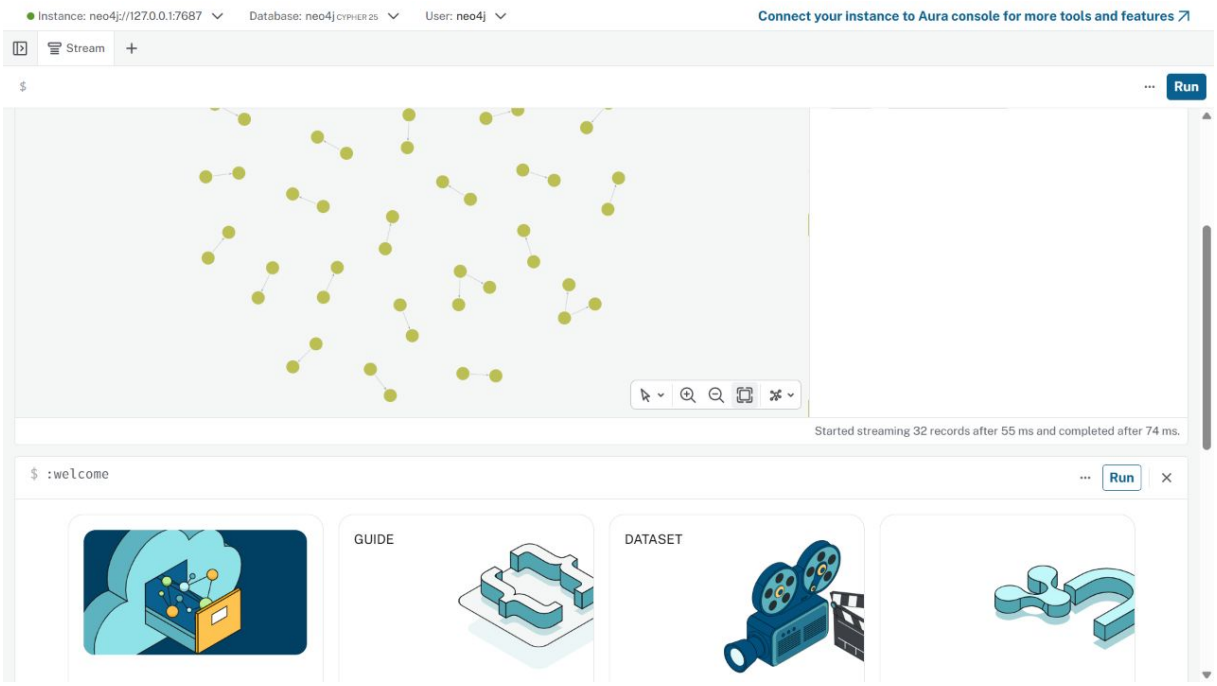
Hướng ưu tiên là mở rộng giao thức agent và bộ nhớ hội thoại, mở rộng tập câu hỏi đánh giá truy xuất độc lập, bổ sung kiểm tra đơn vị có nguồn và cập nhật KB từ tài liệu được kiểm chứng. Sau khi có các thành phần đó mới đánh giá lợi ích phối hợp so với từng thành phần đơn lẻ bằng các phép loại bỏ thành phần trên cùng tập kiểm tra.



Hình 5.1: Suy luận phân loại từ mẫu của bộ dữ liệu.

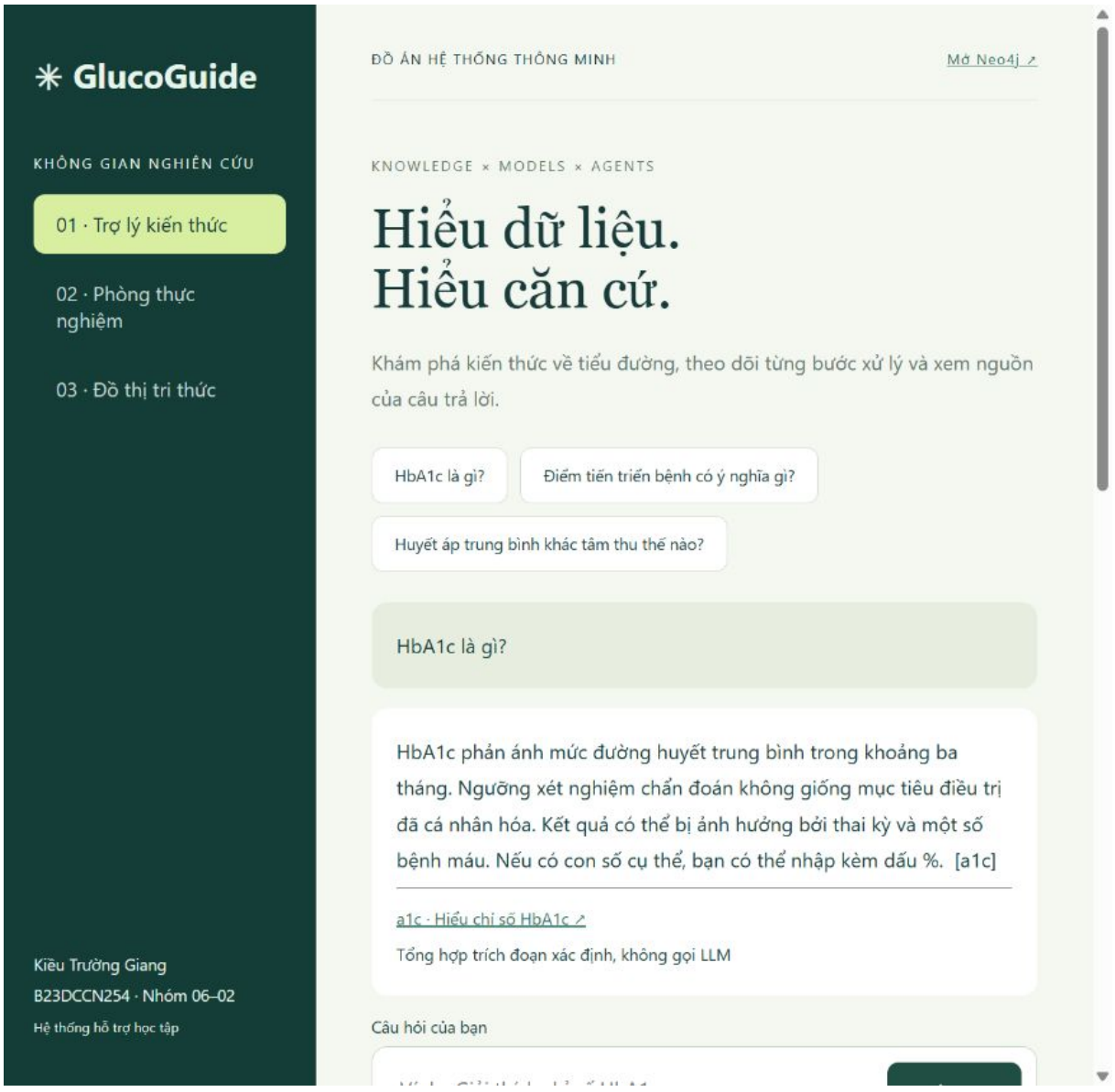


Hình 5.2: Đầu ra hồi quy là điểm tiến triển nghiên cứu.

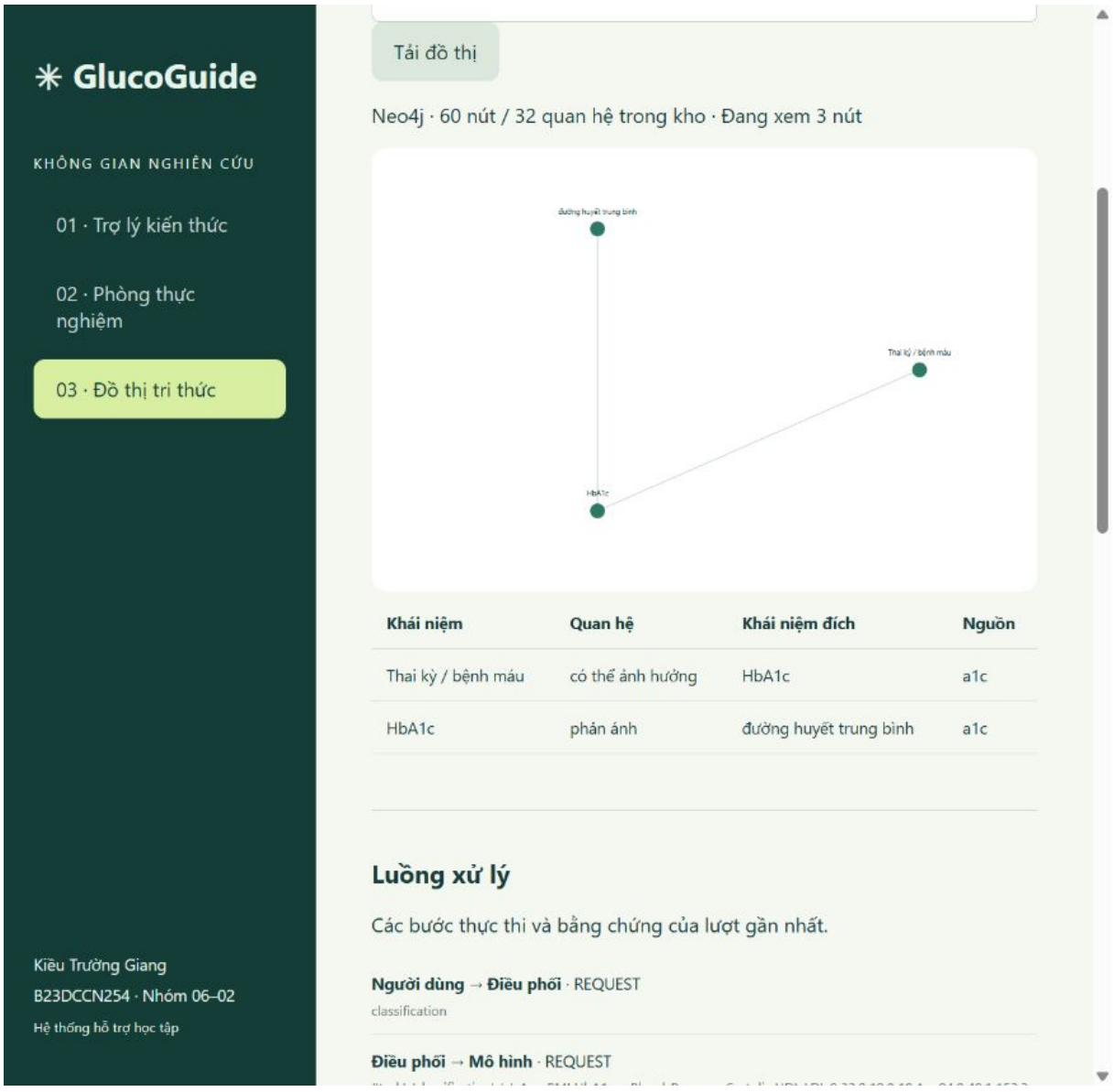


Hình 5.3: Neo4j Browser đã kết nối và trả về các quan hệ Concept.

5.4 Ảnh chụp sản phẩm đang chạy



Hình 5.4: Chatbot trả lời HbA1c kèm mã tài liệu và liên kết nguồn.



Hình 5.5: Lọc quan hệ theo tài liệu a1c, dữ liệu đọc trực tiếp từ Neo4j.

Tài liệu tham khảo

[1] Kiều Trường Giang, Tiểu luận 02, nhóm 06–02, tài liệu học phần do sinh viên cung cấp.

[2] Kiều Trường Giang, Tiểu luận 03, B23DCCN254, tài liệu và thực nghiệm học phần do sinh viên cung cấp.

[3] Phạm Mai Linh, Tiểu luận 01, nhóm 06–03, tài liệu tham khảo do sinh viên chỉ định.

[4] Nguyễn Tự Kiên, Tiểu luận final CT_07 và Project01 CT_07, hai PDF tham khảo về bố cục do sinh viên cung cấp.

[5] T. Kipf và M. Welling, Semi-Supervised Classification with Graph Convolutional Networks, ICLR 2017. https://arxiv.org/abs/1609.02907.

[6] W. Hamilton, R. Ying và J. Leskovec, Inductive Representation Learning on Large Graphs, 2017. https://arxiv.org/abs/1706.02216.

[7] P. Velickovic và cộng sự, Graph Attention Networks, ICLR 2018. https://arxiv.org/abs/1710.10903.

[8] M. Schlichtkrull và cộng sự, Modeling Relational Data with Graph Convolutional Networks, 2017. https://arxiv.org/abs/1703.06103.

[9] Scikit-learn, tài liệu load_diabetes. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_diabetes.html.

[10] Mobeen Fatimah, Diabetes Risk Prediction Dataset 50K Patients. https://www.kaggle.com/datasets/mobeenfatimah/diabetes-risk-prediction-dataset-50k-patients.

[11] NIDDK, Diabetes Tests and Diagnosis. https://www.niddk.nih.gov/health-information/diabetes/overview/tests-diagnosis.

[12] Danh mục nguồn tri thức của project được lưu tại data/knowledge.json; mỗi đoạn văn có mã tài liệu, mã nguồn và các bộ ba có căn cứ.

Phụ lục A

Mã nguồn tái lập

A.1 scripts/ingest_graph.py

```
from pathlib import Path
import json,os
from neo4j import GraphDatabase
root=Path(__file__).resolve().parents[1]
env=dict(x.split('=',1) for x in (root/'.env').read_text().splitlines() if '=' in x)
kb=json.loads((root/'data/knowledge.json').read_text(encoding='utf-8'))
rows=[dict(source=t[0],label=t[1],target=t[2],document=d['id'],evidence=d['text']) for d in
 ↪  kb['documents'] for t in d.get('triples',[])]
with GraphDatabase.driver(env['NEO4J_URI'],auth=(env['NEO4J_USER'],env['NEO4J_PASSWORD']),connection_t
 ↪  imeout=5) as
 ↪  driver:
 driver.execute_query('CREATE CONSTRAINT concept_name IF NOT EXISTS FOR (c:Concept) REQUIRE c.name IS
  ↪  UNIQUE')
 driver.execute_query('UNWIND $rows AS row MERGE (a:Concept {name:row.source}) MERGE (b:Concept
  ↪  {name:row.target}) MERGE (a)-[r:RELATED_TO {label:row.label,document:row.document}]->(b) SET
  ↪  r.evidence=row.evidence',rows=rows)
 print(driver.execute_query('MATCH (n:Concept) RETURN count(n) AS nodes').records)
(root/'results/graph_import.json').write_text(json.dumps({'documents':len(kb['documents']),'relations'
 ↪  :len(rows),'storage':'Neo4j'}))
```

A.2 scripts/train_models.py

```
"""Train reproducible MLP classifier and regression baseline on distinct datasets."""
from pathlib import Path
import json, zipfile, warnings
import numpy as np
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier, MLPRegressor
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.metrics import accuracy_score, f1_score, mean_absolute_error, r2_score
import joblib
ROOT=Path(__file__).resolve().parents[1]
source=ROOT/'data/raw/diabetes.zip'
with zipfile.ZipFile(source) as z:
 df=pd.read_csv(z.open(z.namelist()[0]))
features=['Age','BMI','HbA1c','Fasting_Blood_Sugar','Blood_Glucose','Blood_Pressure_Systolic','HDL','L
 ↪  DL']
```

```
X=df[features];y=df.Diabetes_Risk
tr,te=train_test_split(np.arange(len(df)),test_size=.2,stratify=y,random_state=254)
rows=[]
for name,est in
 ↪  [('LogisticRegression',LogisticRegression(max_iter=1000,class_weight='balanced')),('MLPClassifier'
 ↪  ,MLPClassifier(hidden_layer_sizes=(64,32,16),early_stopping=True,max_iter=150,random_state=254))]:
 model=make_pipeline(SimpleImputer(strategy='median'),StandardScaler(),est)
 model.fit(X.iloc[tr],y.iloc[tr]); pred=model.predict(X.iloc[te])
 rows.append(dict(task='classification',model=name,n_train=len(tr),n_test=len(te),accuracy=accuracy_sc
  ↪  ore(y.iloc[te],pred),macro_f1=f1_score(y.iloc[te],pred,average='macro')))
 joblib.dump(model,ROOT/'models'/f'{name}.joblib')
 if name.startswith('MLP'): pd.DataFrame({'row':te,'actual':y.iloc[te].values,'predicted':pred}).to_cs
  ↪  v(ROOT/'results/classification_predictions.csv',index=False)
d=load_diabetes(scaled=False)
tr,te=train_test_split(np.arange(len(d.target)),test_size=.2,random_state=254)
for name,est in [('Ridge',Ridge(alpha=10)),('MLPRegressor',MLPRegressor(hidden_layer_sizes=(64,32,16),
 ↪  max_iter=1000,early_stopping=True,random_state=254))]:
 model=make_pipeline(StandardScaler(),est);model.fit(d.data[tr],d.target[tr]);pred=model.predict(d.dat
  ↪  a[te])
 rows.append(dict(task='regression',model=name,n_train=len(tr),n_test=len(te),mae=mean_absolute_error(
  ↪  d.target[te],pred),r2=r2_score(d.target[te],pred)))
 joblib.dump(model,ROOT/'models'/f'{name}.joblib')
 if name.startswith('MLP'): pd.DataFrame({'row':te,'actual':d.target[te],'predicted':pred}).to_csv(ROO
  ↪  T/'results/regression_predictions.csv',index=False)
(ROOT/'results/model_metrics.json').write_text(json.dumps(rows,indent=2))
(ROOT/'models/schema.json').write_text(json.dumps({'classification':features,'regression':list(d.featu
 ↪  re_names),'seed':254,'classification_scope':'synthetic dataset
 ↪  labels','regression_scope':'research progression score, not glucose or personal disease
 ↪  probability'},indent=2))
print(json.dumps(rows,indent=2))
```

A.3 scripts/train_policy.py

```
from pathlib import Path
import json,numpy as np
root=Path(__file__).resolve().parents[1]
rng=np.random.default_rng(254);q=np.zeros((4,3));history=[]
# Explicit simulated workflow: missing evidence -> clarify; evidence -> cite;
# prediction plus evidence -> explain model and cite. Not a treatment policy.
for ep in range(3000):
 s=int(rng.integers(4));a=int(rng.integers(3)) if rng.random()<.2 else int(q[s].argmax())
 target=0 if s%2==0 else (2 if s==3 else 1)
 reward=1. if a==target else -1.
 q[s,a]+=.15*(reward-q[s,a]);history.append(reward)
payload={'seed':254,'episodes':3000,'q':q.tolist(),'actions':['Hỏi làm rõ','Trả lời có nguồn','Giải
 ↪  thích mô hình và dẫn nguồn'],'environment':'One-step simulated workflow, not clinical
 ↪  outcomes','last_300_reward':float(np.mean(history[-300:]))}
(root/'models/workflow_policy.json').write_text(json.dumps(payload,ensure_ascii=False,indent=2),encodi
 ↪  ng='utf-8')
print(payload['last_300_reward'])
```

A.4 app/agents.py

```
from dataclasses import dataclass,field
from typing import Callable,Any
@dataclass
class Message:
 sender:str
 receiver:str
 type:str
 content:Any
@dataclass
class Agent:
 name:str
 handler:Callable
 memory:dict=field(default_factory=dict)
 def receive(self,message,bus):
  self.memory['calls']=self.memory.get('calls',0)+1
  self.memory['last_request']=message.type
  return self.handler(message.content,bus)
class MessageBus:
 def __init__(self):self.agents={};self.trace=[]
 def register(self,agent):self.agents[agent.name]=agent
 def request(self,sender,receiver,content,kind='REQUEST'):
  self.trace.append(vars(Message(sender,receiver,kind, str(content) if isinstance(content,dict) and
  ↪  'x' in content else content)))
  try:
   result=self.agents[receiver].receive(Message(sender,receiver,kind,content),self)
   self.trace.append(vars(Message(receiver,sender,'INFORM',result.tolist() if hasattr(result,'tolist')
   ↪  else result)))
   return result
  except Exception as exc:
   self.trace.append(vars(Message(receiver,sender,'FAILURE',type(exc).__name__)))
   raise
```

A.5 app/main.py

```
import json, os, math, re, unicodedata
from pathlib import Path
import numpy as np
import pandas as pd
import joblib
from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, Field
from neo4j import GraphDatabase
from app.agents import Agent, MessageBus
from sklearn.feature_extraction.text import TfidfVectorizer
ROOT=Path(__file__).resolve().parents[1]
for line in (ROOT/'.env').read_text().splitlines() if (ROOT/'.env').exists() else []:
 if '=' in line:
  k,v=line.split('=',1);os.environ.setdefault(k,v)
kb=json.loads((ROOT/'data/knowledge.json').read_text(encoding='utf-8'))
docs=kb['documents']
vec=TfidfVectorizer(strip_accents='unicode',ngram_range=(1,2))
mat=vec.fit_transform([d['title']+' '+' '.join(d['terms'])+' '+d['text'] for d in docs])
schema=json.loads((ROOT/'models/schema.json').read_text())
```

```
models={k:joblib.load(ROOT/'models'/f'{v}.joblib') for k,v in
 ↪  [('classification','MLPClassifier'),('regression','MLPRegressor')]}
app=FastAPI(title='GlucoGuide • Giang KT')
app.mount('/assets',StaticFiles(directory=ROOT/'web'),name='assets')
class Request(BaseModel):
 question:str=Field(default='',max_length=3000)
 task:str='knowledge'
 values:dict=Field(default_factory=dict)
def graph_driver():
 return GraphDatabase.driver(os.environ.get('NEO4J_URI','bolt://127.0.0.1:7687'),auth=(os.environ.get(
  ↪  'NEO4J_USER','neo4j'),os.environ.get('NEO4J_PASSWORD','')),connection_timeout=3,connection_acquis
  ↪  ition_timeout=4,max_transaction_retry_time=0)
@app.get('/')
def home():return FileResponse(ROOT/'web/index.html')
@app.get('/api/metrics')
def metrics():return json.loads((ROOT/'results/model_metrics.json').read_text())
@app.get('/api/schema')
def fields():return schema
@app.get('/api/examples')
def examples():return json.loads((ROOT/'data/examples.json').read_text(encoding='utf-8'))
@app.get('/api/graph')
def graph():
 try:
  with graph_driver() as driver:
   rows=driver.execute_query('MATCH (a:Concept)-[r:RELATED_TO]->(b:Concept) RETURN a.name AS
    ↪  source,b.name AS target,r.label AS label,r.document AS document LIMIT 500').records
   edges=[dict(r) for r in rows]
   return {'storage':'Neo4j','nodes':sorted(set(e[k] for e in edges for k in
    ↪  ['source','target'])),'edges':edges}
 except Exception:raise HTTPException(503,'Neo4j chưa kết nối được. Khởi động cơ sở dữ liệu và kiểm
  ↪  tra cấu hình .env.')
@app.post('/api/chat')
def chat(req:Request):
 bus=MessageBus();trace=bus.trace
 bus.register(Agent("Mô hình",lambda payload,b: models[payload["task"]].predict(payload["x"])[0]))
 bus.register(Agent("Truy xuất",lambda payload,b: (mat@vec.transform([payload]).T).toarray().ravel()))
 bus.register(Agent("Đối chiếu",lambda payload,b: {"scope":payload.get("scope","Kiến thức có
  ↪  nguồn"),"allow_diagnosis":False}))
 bus.register(Agent("Phản hồi",lambda payload,b: "\n\n".join(d["text"]+" ["+d["id"]+"]" for d in
  ↪  payload)))
 def msg(sender,receiver,kind,content):trace.append(dict(sender=sender,receiver=receiver,type=kind,con
  ↪  tent=content))
 msg('Người dùng','Điều phối','REQUEST',req.question or req.task)
 if req.task not in ['knowledge','classification','regression']:raise HTTPException(422,'Bài toán
  ↪  không hợp lệ')
 result=None
 if req.task!='knowledge':
  fields=schema[req.task];missing=[f for f in fields if f not in req.values or req.values[f] in
   ↪  ['',None]]
  if missing:
   msg('Kiểm tra','Phản hồi','REQUEST','Bổ sung: '+', '.join(missing))
   return {'answer':'Cần bổ sung các trường: '+',
    ↪  '.join(missing),'trace':trace,'sources':[],'result':None}
  try:
   values=[float(req.values[f]) for f in fields]
   if not all(math.isfinite(v) for v in values):raise ValueError()
   if any(abs(v)>10000 for v in values):raise ValueError()
  except (ValueError,TypeError):raise HTTPException(422,'Các giá trị phải là số hữu hạn trong miền
   ↪  nhập liệu.')
```

```
 model=models[req.task];x=pd.DataFrame([values],columns=fields) if req.task=='classification' else
  ↪ np.array([values])
 pred=bus.request('Điều phối','Mô hình',{'task':req.task,'x':x})
 if req.task=='classification':
  result={'label':str(pred),'scores':dict(zip(map(str,model.classes_),map(float,model.predict_proba(x
   ↪ )[0]))),'scope':'Nhãn dữ liệu tổng hợp; không phải chẩn đoán hay xác suất mắc
   ↪ bệnh.'}
 else:result={'value':float(pred),'scope':'Điểm tiến triển bệnh nghiên cứu; không phải glucose hoặc
  ↪ HbA1c.'}
 msg('Mô hình','Đối chiếu','INFORM',json.dumps(result,ensure_ascii=False))
q=req.question or ('phân loại nhãn tổng hợp classification High Moderate Low' if
 ↪ req.task=='classification' else 'điểm tiến triển diabetes')
scores=bus.request('Điều phối','Truy xuất',q)
selected=[docs[i] for i in np.argsort(-scores)[:3] if scores[i]>max(.04,float(scores.max())*.65)]
msg('Điều phối','Tri thức','REQUEST',q)
msg('Tri thức','Đối chiếu','INFORM','Đã truy xuất: '+', '.join(d['id'] for d in selected))
graph_evidence=[]
try:
 with graph_driver() as driver:
  graph_evidence=[dict(r) for r in driver.execute_query('MATCH
   ↪ (a:Concept)-[r:RELATED_TO]->(b:Concept) WHERE r.document IN $ids RETURN a.name AS
   ↪ source,r.label AS relation,b.name AS target LIMIT 12',ids=[d['id'] for d in selected]).records]
 msg('Neo4j','Tri thức','INFORM',str(len(graph_evidence))+' quan hệ có nguồn')
except Exception:msg('Neo4j','Tri thức','FAILURE','Không kết nối; câu trả lời chỉ dùng truy xuất văn
 ↪ bản.')
# Learned workflow policy; rules constrain output when no evidence exists.
policy=json.loads((ROOT/'models/workflow_policy.json').read_text(encoding='utf-8'))
state=int(result is not None)*2+int(bool(selected)); action=int(np.argmax(policy['q'][state]))
if not selected:action=0
msg('Chính sách RL','Phản
 ↪ hồi','INFORM',{'state':state,'action':policy['actions'][action],'rule':'Thiếu bằng chứng thì yêu
 ↪ cầu làm rõ'})
answer=bus.request('Điều phối','Phản hồi',selected) if action in (1,2) else 'Tôi chưa tìm thấy bằng
 ↪ chứng phù hợp trong kho tri thức. Bạn hãy nêu rõ chỉ số hoặc chủ đề cần tìm hiểu.'
if result:
 verdict=bus.request('Điều phối','Đối chiếu',result)
 answer=verdict['scope']+'\n\n'+answer
msg('Đối chiếu','Phản hồi','INFORM','Giữ đúng phạm vi mô hình, dẫn nguồn tài liệu; không suy ra chẩn
 ↪ đoán từ nhãn tổng hợp.')
msg('Phản hồi','Người dùng','INFORM','Hoàn thành')
return {'answer':answer,'result':result,'trace':trace,'sources':[{'id':d['id'],'title':d['title'],'so
 ↪ urce':kb.get('sources',{}).get(d.get('source'),{})} for d in
 ↪ selected],'graph_evidence':graph_evidence,'generator':'Tổng hợp trích đoạn xác định, không gọi
 ↪ LLM'}
```

A.6 scripts/test_scenarios.py

```
from pathlib import Path
import json
from fastapi.testclient import TestClient
from unittest.mock import patch
from app.main import app
r=Path(__file__).resolve().parents[1];c=TestClient(app);examples=json.loads((r/'data/examples.json').r
 ↪ ead_text())
```

```
cases=[('knowledge',{'question':'HbA1c là
 ↪  gì?'},200),('unknown',{'question':'zzzxxyy'},200),('classification',{'task':'classification','valu
 ↪  es':examples['classification']},200),('regression',{'task':'regression','values':examples['regress
 ↪  ion']},200),('missing',{'task':'classification','values':{}},200),('invalid',{'task':'classificati
 ↪  on','values':{**examples['classification'],'Age':'nan'}},422),('bad_task',{'task':'unknown'},422)]
results=[]
for name,payload,status in cases:
 response=c.post('/api/chat',json=payload);body=response.json();assert
  ↪  response.status_code==status,(name,body)
 if name=='unknown':assert body['sources']==[]
 if name=='knowledge':assert body['sources'][0]['id']=='a1c' and body['graph_evidence']
 if name=='classification':assert body['result']['label'] in ['High','Moderate','Low']
 if name=='regression':assert isinstance(body['result']['value'],float)
 if name=='missing':assert body['result'] is None
 results.append({'case':name,'status':status,'passed':True,'response':body})
with patch('app.main.graph_driver',side_effect=RuntimeError('controlled outage')):
 response=c.post('/api/chat',json={'question':'HbA1c'});body=response.json();assert
  ↪  response.status_code==200 and any(t['type']=='FAILURE' for t in body['trace'])
 assert c.get('/api/graph').status_code==503
 results.append({'case':'controlled_neo4j_outage','passed':True,'response':body})
g=c.get('/api/graph').json();assert g['storage']=='Neo4j' and len(g['nodes'])>0
(r/'results/api_scenarios.json').write_text(json.dumps(results,ensure_ascii=False,indent=2),encoding='
 ↪  utf-8')
print(json.dumps({'passed':len(results),'nodes':len(g['nodes']),'edges':len(g['edges'])}))
```